



# Sistemas Distribuidos e Internet

Seminario

Sistemas distribuidos más allá de la Web

# ¿Por qué hablar de esto en SDI?

Porque la Web es solo un caso particular.

Muchas aplicaciones distribuidas no usan navegador.

Detrás de TVs, móviles, sensores, videojuegos o colas de trabajo hay distribución.

## Ejemplos

- chat y mensajería instantánea
- domótica e IoT
- monitorización industrial
- videojuegos online
- impresión en red o colas de tareas
- plataformas de video bajo demanda

# ¿Qué es un sistema distribuido?

## Definición simple

Varios procesos o máquinas  
que cooperan por red  
para dar un servicio común

## El usuario idealmente ve...

un único sistema  
aunque por dentro haya  
muchas piezas separadas

## La dificultad real

las piezas se coordinan  
con retrasos, fallos y  
datos/mensajes que pueden  
perderse

## Ejemplos distribuidos no-web

### **Chat**

clientes + servidor  
+ envío de  
mensajes

### **IoT**

sensores + pasarela  
+ broker de  
eventos

### **Videojuego online**

clientes + servidor  
de estado

### **Sistema industrial**

máquinas +  
alarmas +  
monitorización

### **Plataforma bajo demanda**

Streaming de  
video,  
cliente/servidor  
calidad dinámica  
con pérdida

# Modelos de comunicación

## Petición–respuesta

Un proceso pide algo y espera respuesta.  
Ej.: sockets cliente-servidor, RPC.

## Mensajería

Se deja un mensaje en una cola.  
El receptor lo procesa después.  
Ej.: RabbitMQ.

## Publicación–suscripción

Un emisor publica en un tema y varios  
receptores reciben.  
Ej.: MQTT, Kafka.

## Streaming

Los datos viajan como flujo continuo,  
no como mensajes sueltos.  
Ej.: audio, vídeo, telemetría.

## Broadcast / multicast

Un emisor envía a todos o a un grupo.  
Útil en descubrimiento o difusión local.

## Peer-to-peer y eventos

P2P: nodos entre iguales.  
Eventos: importa que 'algo ha ocurrido'.  
Ej.: BitTorrent, buses de eventos.

*Idea para clase: las 3 primeras son las más importantes; las demás suelen verse como variantes o combinaciones.*

# Los problemas básicos de verdad

La red introduce latencia.

Los nodos pueden caer de forma parcial.

Dos procesos pueden modificar a la vez.

Un mensaje puede llegar tarde, duplicado o no llegar.

Mantener el estado coherente no es trivial.

## Moraleja

Programar distribuido no es solo “usar sockets”.  
Hay que diseñar pensando en fallos.

# Reglas prácticas de diseño

Usar timeouts: no esperar indefinidamente.

Prever reintentos cuando tenga sentido.

Diseñar operaciones idempotentes cuando sea posible.

Separar productores y consumidores con colas o eventos.

Registrar logs para entender qué ha pasado.

## Objetivo

No evitar todos los fallos, sino seguir funcionando razonablemente bien.



# Tecnologías que conviene conocer

## Petición- respuesta

- Sockets TCP
- XML-RPC
- gRPC
- Java RMI

## Mensajería / colas

- RabbitMQ
- ActiveMQ
- Amazon SQS
- ZeroMQ

## Publicación- suscripción

- MQTT
- Kafka
- Redis Pub/Sub
- NATS

## Otras variantes

- WebSocket /  
gRPC streaming
- UDP multicast
- BitTorrent /  
libp2p



# MQTT también necesita servidor

En MQTT ese servidor se llama broker: recibe mensajes y los reparte a los suscriptores.

## Idea clave

Los clientes MQTT no suelen hablar entre sí directamente.

Todos se conectan a un broker central.

El broker recibe publicaciones y las entrega según el topic.

Ejemplos reales de broker: Mosquitto, EMQX, HiveMQ.

Aplicaciones típicas: domótica, sensores, monitorización, IoT.

```
# Broker MQTT local (ejemplo con Mosquitto)
mosquitto -v
```

```
# o en Docker
docker run -it -p 1883:1883 eclipse-mosquitto
```

```
# Después los clientes se conectan al broker
cliente A -> broker -> cliente B
```

En RabbitMQ hablamos de cola y consumidor; en MQTT el centro es el broker, que actúa como servidor de mensajería ligera para dispositivos.

# Caso de uso real usando subscripción y eventos

## Proyecto INMERBOT

**Detección de anomalías acústicas humanas en entornos industriales T1 /**

**Monitorización de emociones en entornos laborales mediante la voz T2**

- 1) Un Robot de inspección BostonDynamics Modelo Spot realiza diferentes misiones por la factoria
- 2) T1 – Monitoriza de forma continua el sonido ambiental
- 3) T2 – Interacciona con los trabajadores en diferentes puntos en los cambios de turno (checkin-out)
- 4) T1 – Detecta eventos acústicos humanos anómalos y envía alarga al cloud de INMERBOT
- 5) T2 – Estima el estado emocional del trabajador por la voz y envia estado emocional al cloud de INMERBOT y a una app local

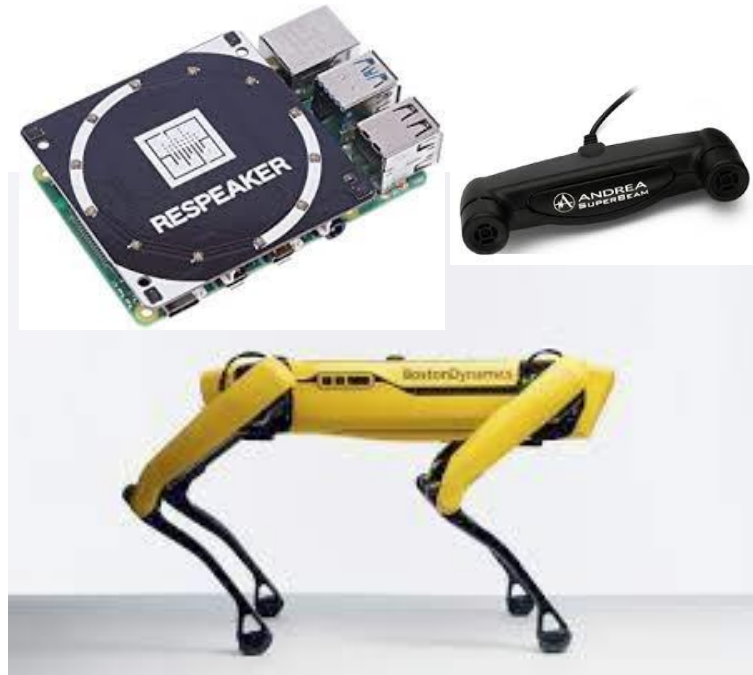
Aquí no hay navegador, pero sí un sistema distribuido completo.

## ➤ Contract on Speech Emotion Recognition in industrial environments

- MIG-2021100, InMerbot: Research In Immersive And Sensory Technologies For Collaborative Industrial Robotic Inspection Environments, Spanish Ministry of Economics and, 2022.
- Principal Investigador: Enrique A. de la Cal
- Suporter: COTESA

## ➤ Challenges

- *Real World Noise*
- *Long Term Emotion Recognition*
- *Data Diversity*

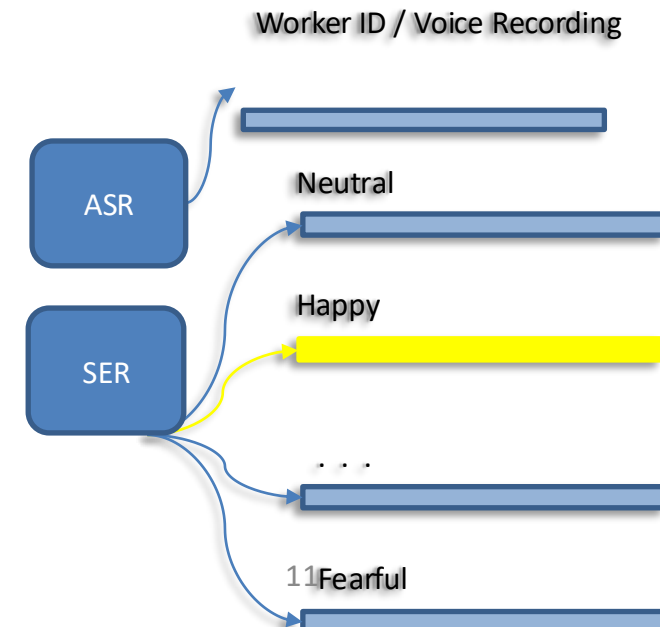


## Goals

- Good Noise Filtering (DSP build-in)
- Voice Personal Identification (OffLine)
- Human-Robot interaction Protocol design

## Current State

- Microphones Tested
- Processing platform selection
  - RPI / Jetson Nano /Xavier



## Fragmentos de código en Python

Ejemplos mínimos con lado cliente y lado servidor, solo para entender la idea.

### Sockets: servidor + cliente

```
# Servidor TCP
import socket
s = socket.socket()
s.bind(("localhost", 5000))
s.listen(1)
conn, _ = s.accept()
print(conn.recv(1024).decode())

# Cliente TCP
import socket
c = socket.socket()
c.connect(("localhost", 5000))
c.sendall(b"HOLA")
```

### RPC: servidor + cliente

```
# Servidor XML-RPC
from xmlrpc.server import SimpleXMLRPCServer
srv = SimpleXMLRPCServer(("localhost", 8000))
srv.register_function(lambda x, y: x + y, "sumar")
srv.serve_forever()

# Cliente XML-RPC
import xmlrpc.client
cli =
xmlrpc.client.ServerProxy("http://localhost:8000")
print(cli.sumar(2, 3))
```

Con sockets tú construyes el canal; con RPC defines operaciones y el cliente llama a funciones remotas.

## Fragmentos de código en Python

Aquí la idea es ver productor/consumidor y publicador/suscriptor.

### RabbitMQ: productor + consumidor

```
# Productor
import pika
ch = pika.BlockingConnection(
    pika.ConnectionParameters("localhost")
).channel()
ch.queue_declare(queue="alarmas")
ch.basic_publish("", "alarmas", b"temperatura alta")

# Consumidor
ch.basic_consume(
    queue="alarmas",
    on_message_callback=lambda c, m, p, b:
print(b.decode())
)
ch.start_consuming()
```

### MQTT: publicador + suscriptor

```
# Publicador
import paho.mqtt.publish as publish
publish.single("edificio/aula3/temp", "27.4",
    hostname="localhost")

# Suscriptor
from paho.mqtt import client as mqtt
cli = mqtt.Client()
cli.on_message = lambda c, u, m: print(m.topic,
m.payload.decode())
cli.connect("localhost", 1883)
cli.subscribe("edificio/aula3/temp")
cli.loop_forever()
```

RabbitMQ deja mensajes en una cola para que otro proceso los consuma; MQTT reparte publicaciones ligeras a quienes est

# Comparativa de tecnologías

| Tecnología             | Modelo                                              | Velocidad / latencia                                      | Complejidad / infraestructura              | Tipo de aplicaciones                                                                       |
|------------------------|-----------------------------------------------------|-----------------------------------------------------------|--------------------------------------------|--------------------------------------------------------------------------------------------|
| Sockets                | Punto a punto comunicación directa                  | Muy rápida latencia muy baja                              | Baja al empezar, pero mucho trabajo manual | Chat, juegos simples, prácticas docentes, comunicación directa entre procesos              |
| RPC / gRPC (o XML-RPC) | Petición-respuesta entre servicios                  | Rápida latencia baja-media                                | Media contratos e interfaces               | Servicios internos, operaciones remotas, consulta de datos, activar funciones en otro nodo |
| RabbitMQ / Kafka       | Mensajería, colas y eventos                         | Buena, pero menos inmediata que socket directo            | Media-alta broker y gestión                | Procesamiento en segundo plano, integración entre sistemas, eventos y monitorización       |
| MQTT                   | Publicación-suscripción ligera                      | Muy eficiente en redes limitadas                          | Media broker sencillo                      | IoT, sensores, domótica, telemetría, dispositivos pequeños                                 |
| Streaming              | Flujo continuo de datos                             | Buena si el flujo es estable; optimizado para tiempo real | Media control del flujo                    | Audio/vídeo en tiempo real, telemetría continua, seguimiento de partidas                   |
| Broadcast / Multicast  | Uno a muchos en red o grupo                         | Muy rápida en red local; depende del soporte              | Media soporte de red                       | Descubrimiento de servicios, avisos en LAN, protocolos de routing, difusión local          |
| Peer-to-peer (P2P)     | Comunicación entre pares sin servidor central claro | Variable según la red                                     | Media-alta coordinación distribuida        | Compartición distribuida de ficheros, blockchain, redes descentralizadas                   |

# Conclusión

Distribuido  $\neq$  solo web.

La comunicación puede ser por llamadas, colas o eventos.

Los problemas clave son red, fallos y coordinación.

Diseñar bien importa más que la tecnología concreta.

## Pregunta final

¿Qué sistema cotidiano usas que probablemente sea distribuido aunque no lo veas?